第5章
函数与编译预处理

中国大学MOOC

学习目标

- 掌握函数的概念、定义和调用方法。
- 理解递归的概念，并能运用递归的方法解决一些实际问题。
- 理解变量的作用域与生存期的概念，能够理解全局变量、局部变量、静态变量的概念和用法。
- 理解编译预处理的概念。
- 了解宏定义、文件包含和条件编译的概念，掌握它们的格式和使用方法。

5.1 函数的基本概念

- 人们在求解一个复杂的问题的时候，通常采用逐步分解、分而治之的方法。也就是把一个大问题分解为几个比较容易求解的小问题，然后分别求解。程序员在设计一个复杂的应用程序时，往往也是把整个程序划分为若干个功能较为单一的程序模块，然后分别予以实现，最后再把所有的程序模块象搭积木一样搭起来，这种在程序设计中分而治之的策略，被称为模块化程序设计方法。
- 在C++中，这些模块就是一个个的函数。

5.1 函数的基本概念

- 函数的本质有两点：
  ① 函数由能完成特定任务的独立程序代码块组成，如有必要，也可调用其它函数，来产生最终的输出。
  ② 函数内部工作对程序的其余部分是不可见的。
- C/C++是由函数构成的，函数是C/C++的基本模块。

中国大学MOOC

5.1 函数的基本概念

- 一个程序必须有且只有一个主函数（main( )函数），和若干个其它函数。 C/C++从main( )函数开始执行，由主函数调用其它函数，其它函数之间也可以互相调用。

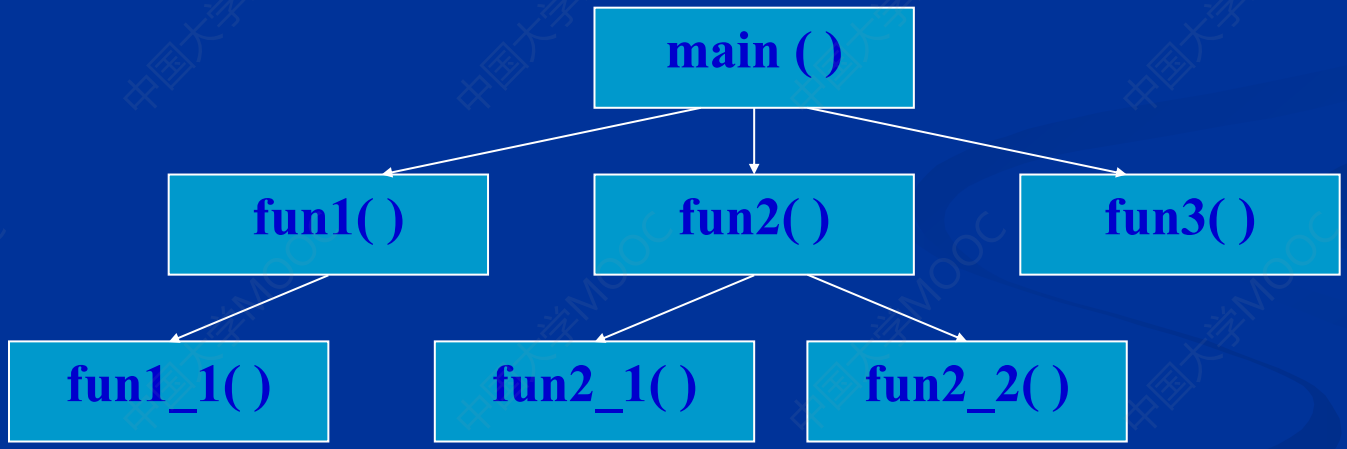


图　函数调用层次关系

5.1 函数的基本概念

1. 库函数

   C/C++系统以提供的函数，提供了丰富的功能。多少随系统而定，有一部分对所有的C/C++系统都有，格式也相同，称为标准函数，还有一部分与系统有关。使用说明:

   A）资料　B）联机帮助

2. 用户自定义函数

   用户根据所需解决的问题而编写

- 函数定义是编写的一段代码（并不执行）
- 函数定义后就可使用它。称为函数调用

中国大学MOOC

5.2 函数的定义

函数定义的一般形式：

返回类型 函数名（参数表列）

{

语句系列

return 合适类型数值

}

说明：

1. 函数返回值指函数返回的值的类型，也有称为：函数类型
2. 函数返回值如果是 void ， 意为无值返回
3. 函数返回值说明若没有，指函数返回整型数。也就是：若函数返回整型类型的数据，可省略函数返回值说明，否则不能省
4. 若参数表空，称为无参函数，否则称为有参函数。
5. 无参函数参数表可写成（void)或写成（）

5.3 函数的调用

要求显示器输出：



在出现函数调用时，程序控制转到函数，函数执行结束后，回到调用点（主函数），继续执行后续语句。

```
using namespace std;
#include <iostream>
#define NUM 11
void print()
{
    int j;
    for (j=0;j<NUM;j++)
        cout<<'*';
    cout<<endl;
}
int main()
{
    print();
    cout<<"How do you do !"<<endl;
    print();
    return 0;
}
```

函数定义

函数调用

单选题 1分

设置

对于C++的非成员函数，下面哪一种说法是错误的？

A 函数的定义不能嵌套，但函数的调用可以嵌套。

B 函数的定义可以嵌套，但函数的调用不能嵌套。

C 一个程序执行从main 函数开始。

D main 函数可调用其它函数，而其它函数一般不调用main 函数。

提交

中国大学MOOC

5.3 函数的调用

```
using namespace std;
#include <iostream>
int max (int a,int b)
{
   if(a> b) return a;
   else return b;
}
```

```
int main()
{
   int  a,b,c, t;
   cin>>a>>b>>c;
   t=max(a,b);
   t=max(t,c);
   cout<<t<<endl;
   cout<<max(a,b+10)<<endl;
   return 0;
}
```

注意：形参与值参数原则上个数、类型应该相一致（特殊情况除外）

若从键盘输入： 9 4 23<Enter>
输出是： ?

【例】说明实参和形参对应关系的示例。

```
using namespace std;
#include <iostream>
#include<cmath>
float power(float x,int n){
//求x的n次幂
  float pow=1;
  while(n--)  pow*=x;
  return pow; }
int main(){
  int n=3;
  float x=4.6;
  char c='a';
  cout<<"power("<<x<<','<<n<<")="<<power(x,n)<<endl;
  cout<<"power("<<c<<','<<n<<")="<<power(c,n)<<endl;
  cout<<"power("<<n<<','<<x<<")="<<power(n,x)<<endl;
  return 0;
 }
```

单选题 1分

设置

对于函数void f(int x)，下面哪一个调用是正确的？

A `int y = f(9);`

B `f(9);`

C `f(f(9));`

D `x = f();`

提交

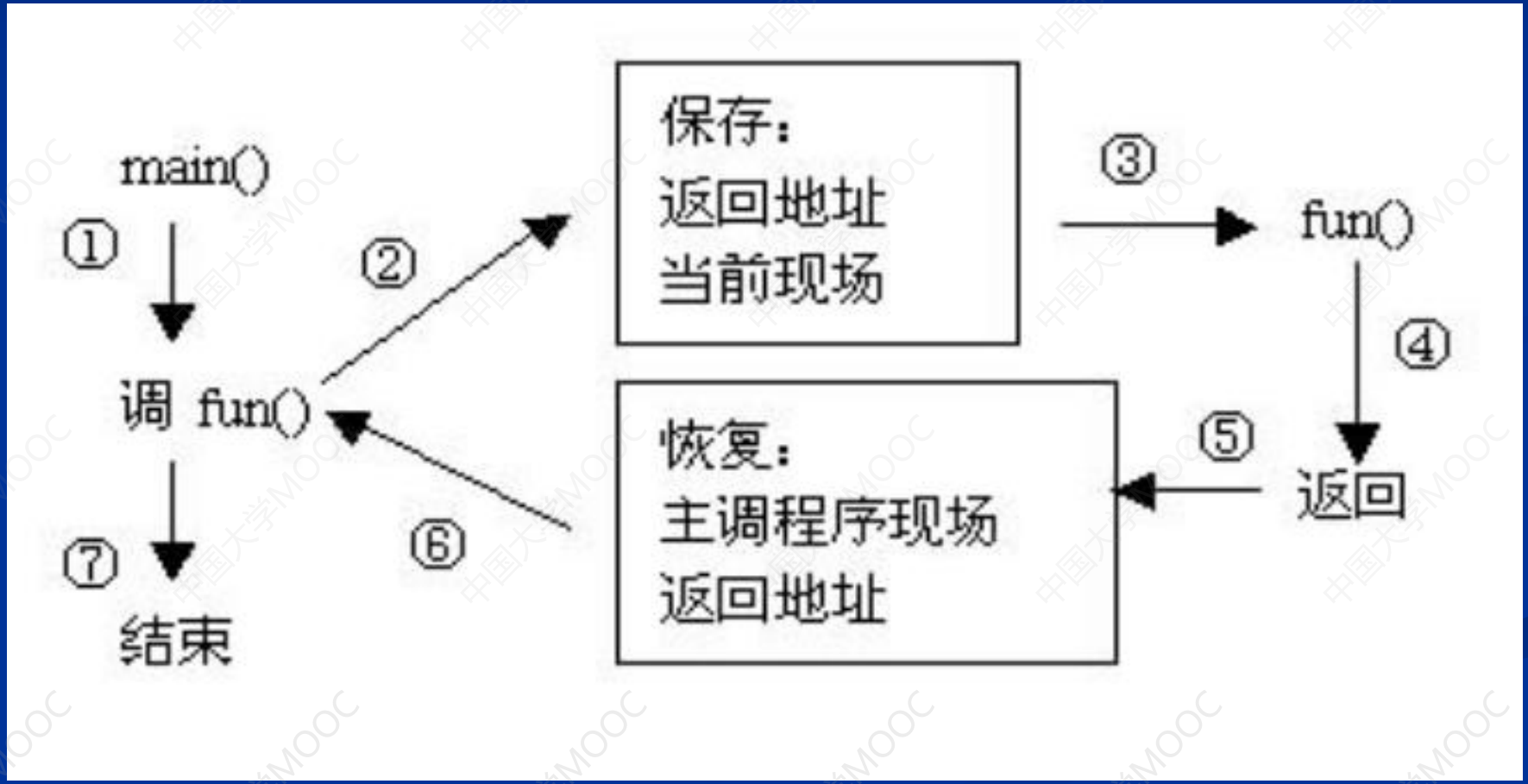
5.3 函数的调用

```
using namespace std;
#include <iostream>
void  change(int x,int y)
{
    int tmp;
    tmp=y;
    y=x;
    x=tmp;
    cout<<"x="<<x<<"y="<<y<<endl;
}

int main()
{  int  x=3,y=4;
    cout<<"x="<<x<<"  x="<<y<<endl;
    change(x,y);
    cout<<"x="<<x<<"  y="<<y<<endl;
    return 0;
}
```

- 在C/C++中，函数的参数传递都是值传递

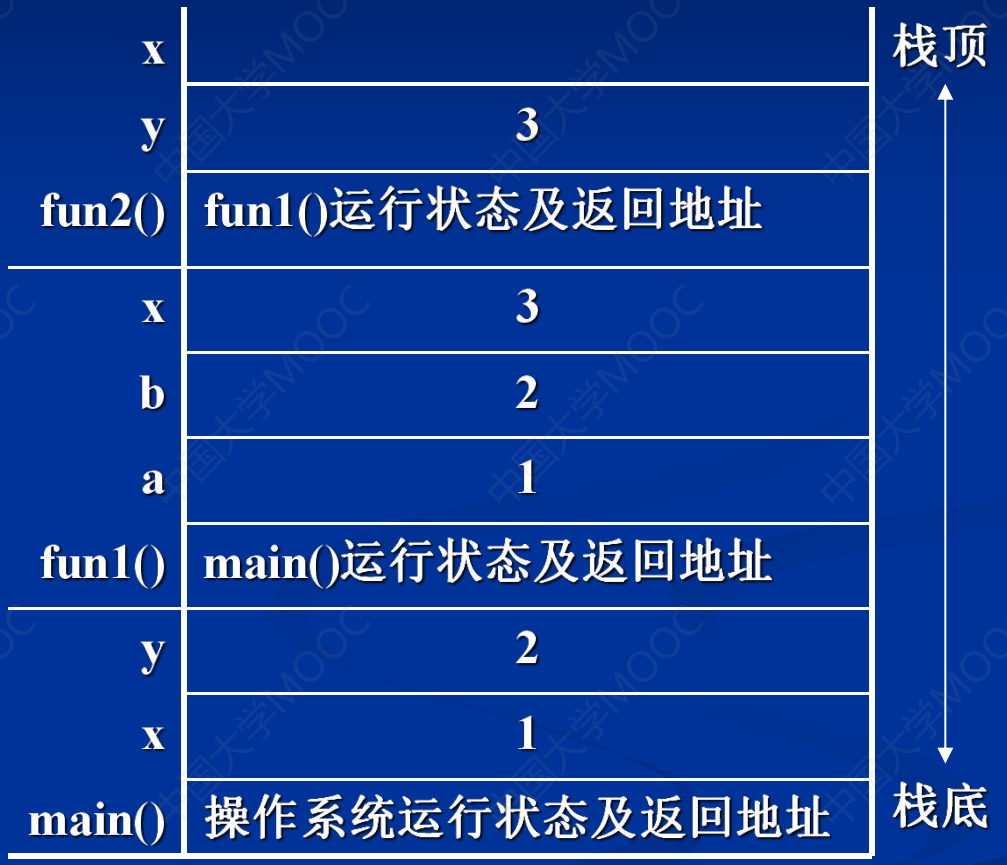
5.3 函数的调用



5.3 函数的调用

- 在每次调用一个函数事件，系统内部产生一个栈架（栈：插入删除只允许在一端进行的一种表），将函数执行结束后的位置记下，然后依此计算实参表达式并将结果放在栈架中，函数内说明的变量也在栈架中得到空间，函数在执行相应的代码时，在栈架中取对应的数据或变量。函数结束后，从栈架中取到返回的位置，并退去本函数此次的栈架。

```
void fun1(int, int);
void fun2(float);
void main(){
    int x=1;y=2;
    fun1(x, y);
}
void fun1(int a,int b){
    float x=3;
    fun2(x);
}
void fun2(float y) {
     int x;
     …
}
```



```
using namespace std;
#include <iostream>
void  change(int x,int y)
{
   int tmp;
   tmp=y;
   y=x;
   x=tmp;
   cout<<"x="<<x<<"y="<<y<<endl;
}

int main()
{  int  x=3,y=4;
   cout<<"x="<<x<<"  x="<<y<<endl;
   change(x,y);
   cout<<"x="<<x<<"  y="<<y<<endl;
   return 0;
}
```

main栈架

main的返回地址
3(x)
4(y)
change的返回地址
4(x)
3(y)
4(tmp)

子程序栈架

5.3 函数的调用

1. 计算实际参数表中各表达式
2. 将表达式的值依此赋给各形式参数
3. 执行函数体
4. 当遇到return语句时，将控制返回到调用函数；
5. 当return语句中包含有表达式时，则计算表达式返回值
6. 当无return语句时，则执行到函数体末后返回调用函数
7. 参数以传值方式调用

投票 最多可选1项 设置

编程练习-素数的判定

题目内容：
编程判定素数。
要求：
编写函数bool IsPrime(int n)判断n是否为素数，若是返回true，否则返回false。在main中通过调用IsPrime函数实现程序功能：若输入的是素数，则输出Yes，否则输出No。
输入样例1：
8
输出样例1：
No
输入样例2：
5
输出样例2：
Yes

A 完成

B 未完成

提交

投票 最多可选1项 设置

编程练习-哥德巴赫猜想

题目内容：

编程将任意一个大正偶数都能分解为两个素数之和，例如，6=3+3、8=3+5。

要求：

编写函数bool IsPrime(int n)判断n是否为素数，若是返回true，否则返回false。在main中通过调用IsPrime函数实现程序功能：输入整数n，若n不是大于2的正偶数，则报错；否则，输出分解的结果。

输入样例1：

8

输出样例1：

8=3+5

输入样例2：

5

输出样例2：

It's not an even larger than 2!

A 测试通过

B 测试未通过

提交

5.4 函数的重载

- 函数的重载是指完成不同功能的函数可以具有相同的函数名。（C语言无此功能）
- 调用重载函数时，C++编译器根据实参的类型或实参的个数来确定应该调用哪一个函数。
- 函数重载体现了C++语言“同一接口，多种实现方法”的多态性机制。
- 注意：
  - 重载函数的参数个数或参数类型必须有不同
  - 仅返回值类型不同时，不能定义为重载函数

5.4 函数的重载

```
using namespace std;
#include<iostream>
sum(int a,int b){
        return a+b;
}
double sum(double a,double b){
        return a+b;
}
float sum(float a,float b,float c){
        return a+b+c;
}
int main(){
        cout<<"3+5="<<sum(3,5)<<endl;
        cout<<"2.2+5.6="<<sum(2.2,5.6)<<endl;
        cout<<"3.5+4+8="<<sum(3.5,4,8)<<endl;
        return 0;
}
```

单选题 1分

设置

下面哪一对函数满足函数重载规则？

A

```
float fun(float x);
void fun(float y);
```

B

```
float funa(float x);
void fun(float x, float y);
```

C

```
float fun(float y);
void fun(float x);
```

D

```
float fun(float x, float y);
void fun(float y);
```

提交

中国大学MOOC

5.5 嵌套调用与递归调用

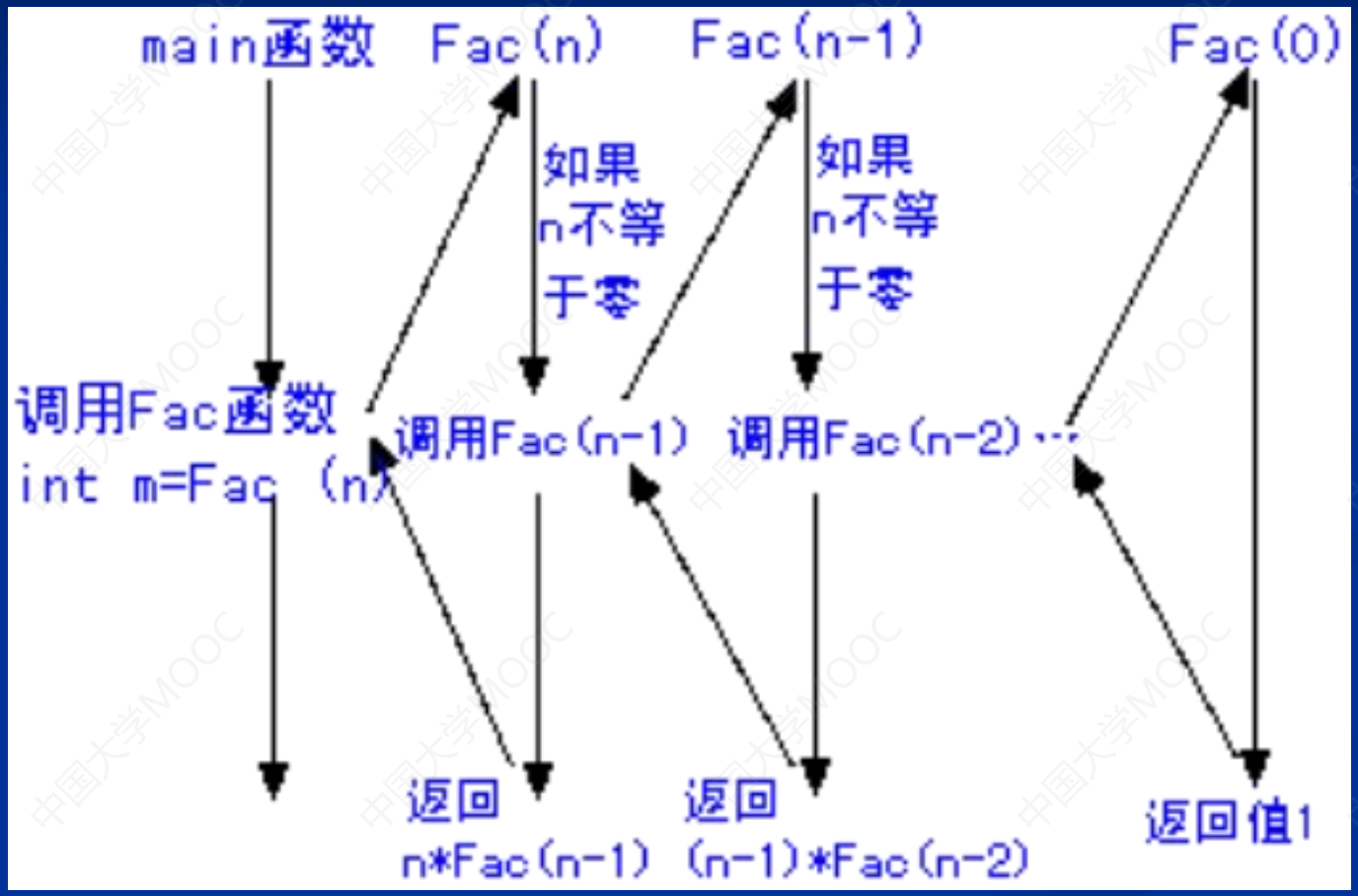
- C语言中，所有函数都是平行独立的，无主次、相互包含之分。函数可以嵌套调用，不可嵌套定义。
- 函数的递归调用：若在函数定义时，调用了自己，称递归调用

5.5 嵌套调用与递归调用

$$n!=\begin{cases}1 & n=1\\ (n-1)!*n & n>1\end{cases}$$

```
using namespace std;
#include <iostream>
unsigned long  fac(unsigned long n)
{
   if( n ==1 ) return 1;
   else return n*fac(n-1);
}
int main()
{    unsigned long n;
     cin>>n;
     cout<<fac(n)<<endl;
     return 0;
}
```

5.5 嵌套调用与递归调用



递归是一种编程技术，并不是用于计算阶乘的。用非递归的方法计算阶乘效率更高，我们用递归来计算阶乘，是因为它的递归调用易于实现和理解。

5.5 嵌套调用与递归调用

- 注意：
  - 编写递归的函数时，一定注意要有递归调用终止的条件，且每调用一次就向调用终止更靠近一步。 这可确保递归调用能够正常结束，而不至于导致系统内存耗尽而崩溃。
  - 递归函数总比功能相同的非递归函数慢，因为每次调用函数时都有调用开销。任何用递归编写 的函数都可使用循环结构代替，以消除递归。

Hanoi塔



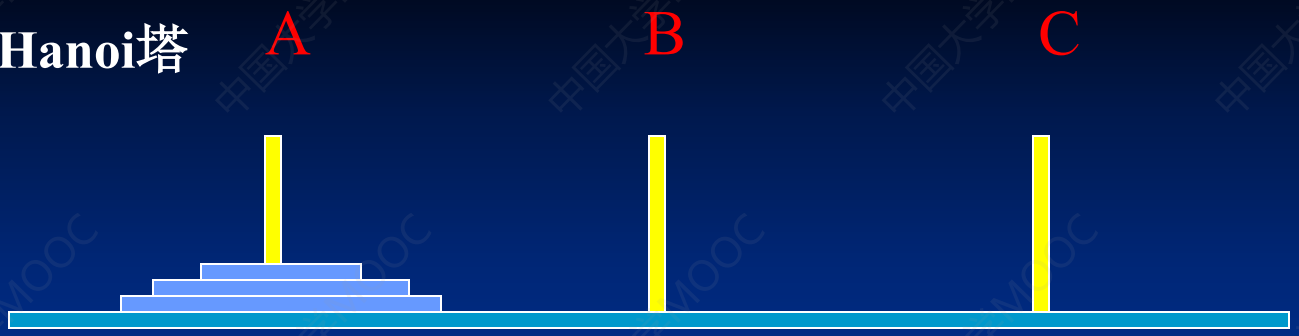

```
hanoi(n,a,b,c)  //n个盘a从移到c，允许利用的中间杆是b
{
 hanoi(n-1,a,c,b);
 a ——→ c;            // cout<<a<<“->”<<c<<endl;
hanoi(n-1,b,a,c);
}
```

当只有一个盘时，上述不成立。应直接从a移到c

```
using namespace std;
#include  <iostream>
void  hanoi(int n,char a,char b,char c)
{
  if(n == 1) cout<<a<<"->"<<c<<endl;
  else {  hanoi(n-1,a,c,b);
          cout<<a<<"->"<<c<<endl;
          hanoi(n-1,b,a,c);}
}
```

```
int main()
{ int n;
  cin>>n;
 hanoi(n,'A','B','C');
 return 0;
}
```

Hanoi塔




```
using namespace std;
#include  <iostream>
void  hanoi(int n,char a,char b,char c)
{
  if(n == 1) cout<<a<<"->"<<c<<endl;
  else {  hanoi(n-1,a,c,b);
          cout<<a<<"->"<<c<<endl;
          hanoi(n-1,b,a,c);}
}
  hanoi(3,'A','B','C');
```

```
void  hanoi(3, 'A', 'B', 'C')
{
  if(n == 1) cout<<a<<"->"<<c<<endl;
  else {  hanoi(n-1,a,c,b);
          cout<<a<<"->"<<c<<endl;
          hanoi(n-1,b,a,c);}
}
```

```
void  hanoi(3,'A', 'B', 'C')
{
   hanoi(2, 'A', 'C', 'B');
  A搬到C;
  hanoi(2, 'B', 'A', 'C');
}
```

n个盘子要移动多少次？

```
unsigned hanoiCount(int n){//n个盘子要移动的次数
      if (n == 1)
            return 1;
      else
            return hanoiCount(n - 1) * 2 + 1;
}

unsigned hanoiCount2(int n){
      if (n < 32)
            return (1 << n) - 1;      //n个1位
      if (n == 32)
            return -1;      //unsigned的最大值为全
1位
      return 0;
}
```

投票 最多可选1项 设置

编程练习-Hanio塔问题

题目内容：
对于Hanoi塔问题，如果要移动n(n<=64)个盘子，假设一秒钟移动1000次，求出完成移动大约需要几年几月几日。按照一年360天，一个月30天来算即可。不足1日的，按1日来算。
输出格式：a b c,其中a代表年，b代表月，c代表日，按照一年360天来算即可。注意行末不要有多余空格。
输入样例1：
1
输出样例1：
0 0 1
输入样例2：
36
输出样例2：
2 2 6

A 测试通过

B 测试未通过

提交

递归例题1

- 小母牛问题：有一头小母牛，从第4年开始每年生一头小母牛，请问第n年共有多少母牛？

```
int cow(int n)
{
    int i,n1=1,n2=1,n3=1,num=1;
    if(n<4)return 1;
    for(i=4;i<=n;i++)
    {
        num=n3+n1;
        n1=n2;
        n2=n3;
        n3=num;
    }
    return num;
}
```

```
#include<iostream>
using namespace std;
int cow(int n)
{
    if(n<=3) return 1;
    else return (cow(n-1)+cow(n-3));
}
int main()
{
  int n;
  cout<<"请输入年份： ";
  cin>>n;
  cout<<"母牛的个数为： "<<cow(n)<<endl;
  return 0;
}
```

填空题 1分

设置

计算3个A，2个B可以组成多少种排列的问题（如：AAABB, AABBA）是《组合数学》的研究领域。请问m个A，n个B可以组合成多少个不同排列？

```
int f(int m, int n)
{
        if(m==0 || n==0) return 1;
        return _____ [填空1] _____;
}
```

正常使用填空题需3.0以上版本雨课堂

作答

填空题 3分

设置

王果果小朋友走楼梯时特别淘气，每次只走2级或3级，请问她走一个n级楼梯，有多少种走法？

```
int f(int n)
{
        if(n==1) return 0;
        if(n== [填空1] || n== [填空2] ) return 1;
        return _____ [填空3] _____;
}
```

正常使用填空题需3.0以上版本雨课堂

作答

中国大学MOOC

单选题 1分

设置

下列代码执行结果是。

```
int f(int Int){
  if (Int==0) return 1;
  return (Int + f(Int-1));
};
int main(void){
  int inT=9;
  cout<< "result=" << f(inT) <<'\n';
  return 0;
}
```

A result=1

B result=37

C result=46

D 编译错

提交

中国大学MOOC

填空题 1分

设置

```
using namespace std;
#include <iostream>
void p(int m, int n){
  if(m>n){
    while(m%n) n++;
          p(m/=n,n);
          cout<<n<<" ";
          }
}
int main(){
          p(420,3);
          return 0;
}
```

以上程序的输出是： [填空1]

作答

投票 最多可选1项 设置

编程练习-逐位正序和反序输出

题目内容：
把输入的正整数实现逐位正序和反序输出。如输入一个整数8597，则输出8597和7958。提示：用递归实现。
要求：
编写函数void original(int n)将参数n 逐位正序输出（注意中间有空格）；编写函数void inverted(int n)将参数n逐位逆序输出（注意中间有空格）；在main中通过调用函数来实现程序功能。
输入样例：
56835
输出样例：
5 6 8 3 5
5 3 8 6 5

A 测试通过

B 测试未通过

提交

5.6 函数原型

```
using namespace std;
#include  <iostream>
 int max (int,int)

int main ( )
{ int a,b,t;
 void print (int,int);
 void print(int,int);
 int max(int, int);
 cin>>a>>b;
 print(a,b);
 t=max(a,b);
 cout<<t<<endl;
 return 0;}
```

```
void  print(int a,int b)
{
     int max (int,int);
  for(i=0;i<max(a,b);i++)
     cout<<‘*’;
  cout<<endl;
 }

max(int x,int y)
{
  return  (x>y)?x:y;
 }
```

- 函数原型，函数若还没有定义，就先使用，就需要给出函数原型，此时只要提供函数返回值类型，函数名，形参类型即可。

5.7 auto函数与尾随返回类型

用auto替代函数返回类型，让编译器自动推导其返回类型。

根据函数体中return表达式来推导函数返回类型。

如果auto函数中有多个return表达式，应为相同类型。

auto函数可以是递归函数，但递归调用必须在至少一个return语句之后。

```
auto fun(int i){
    if(i==1) return 1;
    return (func(i-1)+i);   }
```

auto函数的原型说明可用尾随返回类型

在形参表之后用“->返回类型”说明

```
auto func(int i) ->decltype(i);
```

单选题 1分

设置

假设有

auto func(){auto a = 2u; auto b = 3L; return a + b; }

那么auto v = func(); v 的类型是

A int

B unsigned int

C long

D unsigned long

提交

中国大学MOOC

5.8.1 带默认值的形参

- 在C++中定义函数时，允许给参数指定一个缺省的值。在调用函数时，若明确给出了这种实参的值，则使用相应实参的值；若没有给出相应的实参，则使用缺省的值（C无此功能）。

```
using namespace std;
#include  <iostream>
float fact(int n = 10)
{
    float f=1;
    for ( int i=2; i<=n; i++) f *=i;
    return f;
}
int main(void)
{
    cout << fact(5) <<'\n';
    cout << fact() <<'\n';
}
```

5.8.1 带默认值的形参

- 注意：
  - 必须在函数调用前指定缺省参数的值
  - 参数的缺省值可以是常量表达式，表达式中使用的量必须是常量
  - 具有缺省值的函数参数可有多个，但缺省参数必须位于参数表中的最右边
  - 同一个具有缺省参数的函数，在不同地方调用时可使用不同的缺省参数值

输入长方体的长度，宽度和高度，求出长方体的体积。

```
using namespace std;
#include  <iostream>
float  v(float, float=10, float=20);
int main(void)
{
    float x, y, z;
    cout<<"输入第一个长方体的长、宽、高：";
    cin>>x>>y>>z;
    cout<<"第一个长方体的体积为："<< v(x,y,z) <<'\n';
    cout<<"输入第二个长方体的长和宽：";
    cin>>x>>y;
    cout<<"第二个长方体的体积为："<< v(x,y) <<'\n';
    cout<<"输入第三个长方体的长度：";
    cin>>x;
    cout<<"第三个长方体的体积为："<< v(x) <<'\n';
     return 0;
}

float  v(float a, float b, float c)
{   return a*b*c;  }
```

例： 对于同一个函数，可在不同的作用域内作不同的原型说明，定义不同的缺省参数值。

```
float fact(int n =10);
...

void  bb(void)
{
        float fact(int =20);
        ...
        x=fact( );
        ...
}

float cc (void)
{
        float fact(int =30);
        ...
        y=fact( );
        ...
}

float dd(void)
{
        ...
        z=fact( );
        ...
}

float fact(int n)
{
        float f=1;
        for (int i=2; i<=n; i++) f *=i;
        return f;
}
```

函数的原型说明

参数的缺省值为20

参数的缺省值为30

参数的缺省值为10

函数的定义

5.9 inline函数与constexpr函数

- inline函数称为内联函数。
  - 建议编译器将函数体的代码直接插入到函数调用处，调用函数实质上是顺序执行直接插入的函数代码。
  - 以空间换时间。
  - 仅适合短小函数。

```
inline float max(float x, float y){
    return (x > y ? x : y);
}
```

5.9 inline函数与constexpr函数

读入一行字符串，逐个判断是否为数字字符：

```
using namespace std;
# include <iostream>
inline bool IsNumber(char ch) {
  return ch>='0'&&ch<='9'?true:false;
}
int main(){
    char ch;
    while(cin.get(ch),ch!= '\n'){
      if (IsNumber(ch)) cout<<"是数字字符"<<endl;
      else cout<<"不是数字字符"<<endl;
    }
 }
```

因使用频度很高，说明为内联函数。

5.9 inline函数与constexpr函数

- constexpr函数
  - C++11引入constexpr限定符，常量表达式，表示编译常量，即编译期确定的常量值。
  - constexpr可限定类、函数和变量。
  - constexpr函数在条件满足且确实需要时，可提前到编译期计算，同时也隐含为内联inline函数，执行效率高。

5.10 作用域

- 在C++中，标识符（如变量名）都有确定的说明位置，这决定了它的有效的范围（称为作用域）。
- 作用域分为5类：块作用域、文件作用域、函数原型作用域、函数作用域和类作用域。

5.10.2 文件作用域

文件作用域规则：

1. 在所有函数外说明的变量，从定义变量的位置开始到该程序文件结束，所有函数中均可使用，称为全局变量，全局变量的初值是0
2. 在块作用域内可通过作用域运算符“::”来使用与局部变量同名的全局变量

```
using namespace std;
#include <iostream>
int  i= 100; //全局变量
int main(void)
{
    int i,j=50;    //局部变量
    i=18;
    ::i= ::i+4;
    j= ::i+i;
    cout<<"::i="<<::i<<'\n';
    cout<<"i="<<i<<'\n';
    cout<<"j="<<j<<'\n';
}
```

5.10.2 文件作用域

全局变量的例子

```
using namespace std;
#include<iostream>
int n=100;
void func(){
        n*=2;
}
int main(){
        n*=2;
        cout<<n<<endl;
        func();
        cout<<n<<endl;
 }
```

5.10.2 文件作用域

局部变量的例子

```
using namespace std;
#include<iostream>
void fun(){
    int t=5;
    // fun()中的局部变量
    cout<<"fun()中的t="<<t<<endl;
}
int main(){
    float t=3.5;
    //main()函数中的局部变量
    cout<<"main()中的t="<<t<<endl;
    fun();
    cout<<"main()中的t="<<t<<endl;
}
```

5.10.2 文件作用域

- 在函数内说明的函数原型仅对本函数有效
- 在函数外说明的函数原型对本文件有效

中国大学MOOC

5.10.1 块作用域

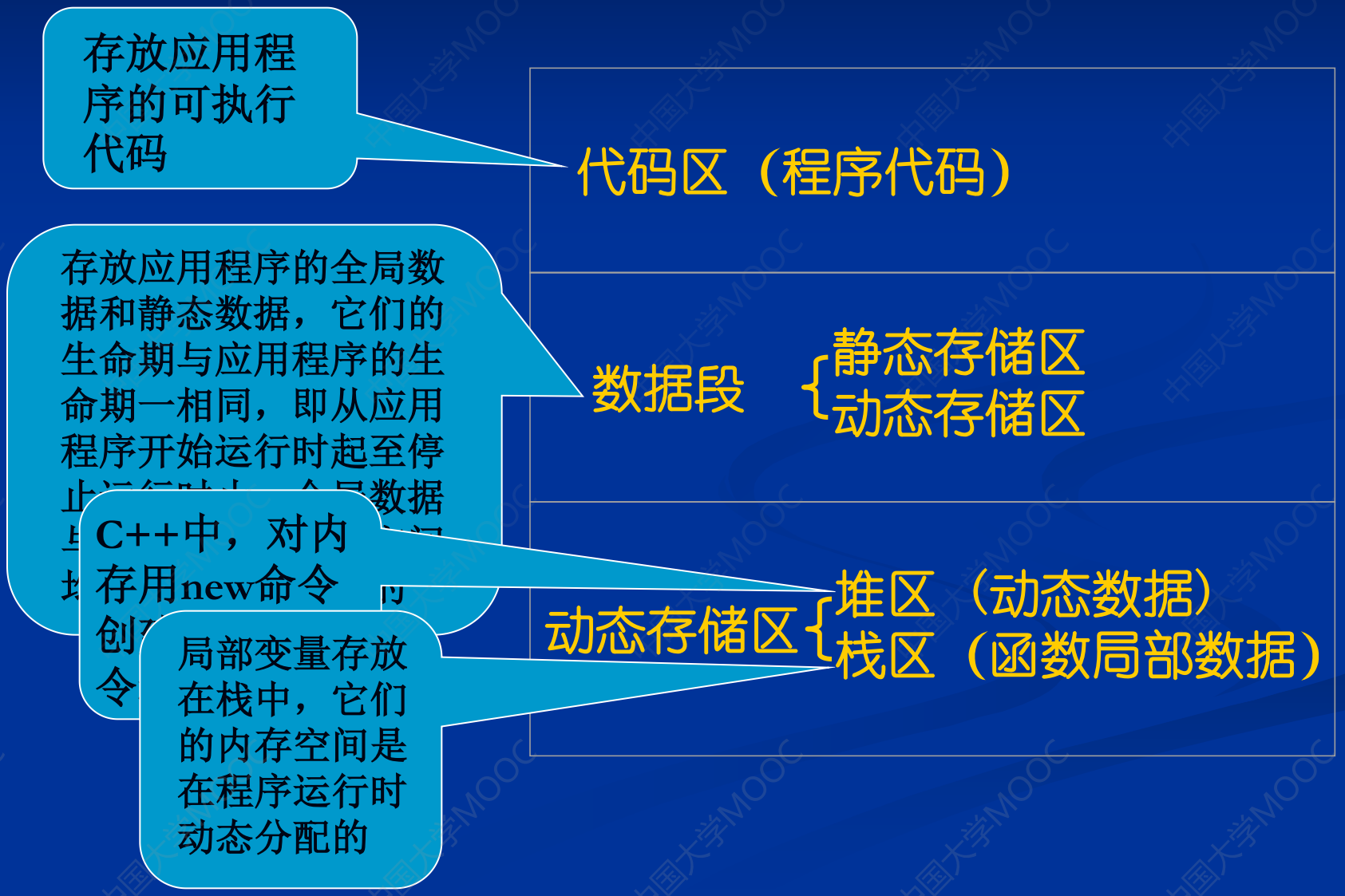
- 一个块（block）就是一条复合语句，用一对花括号{ }括起来的部分。
- 标识符只能在说明它的块内有效，在块之外是不可见的。每个块内可以有各自的命名法，外层块说明的名字如果在内层没有重新定义则在内层中继续有效；一旦内层中重新定义，则内层使用的是内层重新定义了的名字。
- For语句中说明的循环控制变量作用域为包含for语句的那个内层块。

```
int n=100;
using namespace std;
#include<iostream>
int main(){
int i=200,j=300;
cout<< n<<'\t'<<i<<'\t'<<j<<endl;
 {      //内部块
  int i=500,j=600,n;
  n=i+j;
  cout<< n<<'\t'<<i<<'\t'<<j<< endl;
  //输出局部变量n
  cout<<::n<<endl;//输出全局变量n
 }
n=i+j;                    //修改全局变量
cout<< n<<'\t'<<i<<'\t'<<j<< endl;
}
```

```
100     200     300
1100    500     600

500     200     300
```

5.11 程序运行期存储区域



5.12 存储类

- 存储类：规定了变量的生存期，确定何时为变量分配内存空间及何时收回该空间
  - 变量说明：<存储类型> <类型> 标识符;
- 存储类可以是下面四种之一：
  - 自动的 auto
  - 外部的 extern
  - 寄存器的 register
  - 静态的 static

5.12 存储类

- 自动类变量： auto
  - 按缺省规则，不加存储类说明的为自动的。
  - 自动变量进入分程序分配空间（在栈顶），退出分程序，空间由系统回收(退栈）
- 寄存器的 register
  - 变量分配在寄存器中，以加快存储速度
  - 当资源限制做不到时，系统视为auto

5.12 存储类

- 静态的　　static
  - 静态变量存储在静态存储区。
  - 静态变量在程序开始执行时获得所分配的内存，生存期是全程的，作用域是局部的。
  - 静态变量的初始化仅在分配空间时做一次，若不初始化则系统自动置为0。
  - 程序结束时静态变量消亡。
  - 退出函数后仍保留其值，但函数外不可见。

5.12.3 static变量与多文件项目

局部静态变量和局部变量的对比

```
using namespace std;
#include<iostream>
st(){
  static int t=100; //局部静态变量
  t++; return t;
}
at(){
  int t=100; //自动变量
  t++;return t;
}
int main(){
  int i;
  for(i=0;i<5;i++)  cout<<at()<<'\t';
  cout<<endl;
  for(i=0;i<5;i++)  cout<<st()<<'\t';
  cout<<endl;
}
```

5.12.3 static变量与多文件项目

局部静态变量和局部变量的对比

```
using namespace std;
#include<iostream>
st(){
  static int t=100; //局部静态变量
  t++; return t;
}
at(){
  int t=100; //自动变量
  t++;return t;
}
int main(){
  int i;
  for(i=0;i<5;i++)  cout<<at()<<'\t';
  cout<<endl;
  for(i=0;i<5;i++)  cout<<st()<<'\t';
  cout<<endl;
}
```

5.12.3 static变量与多文件项目

- 当static加在全局变量或函数前，则使该变量或函数仅在本文件内有效。
- 见p92

单选题 1分

设置

下面代码运行结果是

```
int t(void)
{ static int i = 1; i += 2; return i; }
int t1(void)
{ int j =1; j += 2; return j; }
int main(void){
  int j=2;
  t(); cout << "I="<<t( )<<" ";
  t1(); cout << "J="<<t1( )<< '\n';
  return 0;
}
```

A I=5 J=3

B I=5 J=5

C I=3 J=5

D I=3 J=-3

提交

中国大学MOOC

5.12.3 static变量与多文件项目

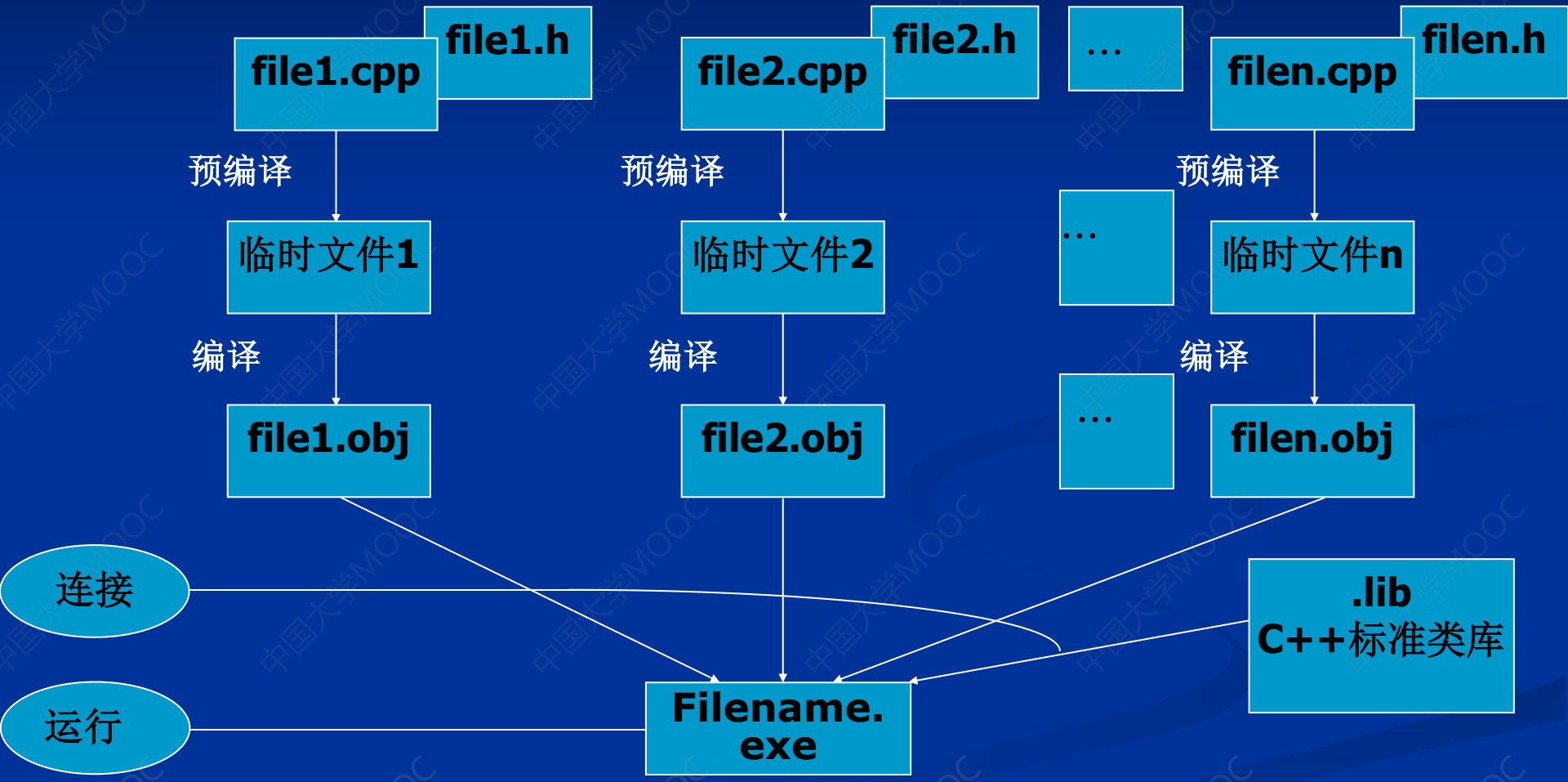


图 C++程序开发过程

5.12 存储类

- 外部类变量： extern
  - 在函数外面说明的变量称外部变量。外部变量的空间是固定的（存储在全局变量区），在程序运行其间一直有效，在程序运行完毕后由操作系统回收。
  - 若使用的全局变量先使用后定义或不在本文件中定义，需要使用则要在使用前加 extern的说明。

5.12.4 extern变量

假定程序包含两个源程序文件Ex5_11_1.cpp和Ex5_11_2.cpp，程序结构如下：

```
/* Ex5_11_1.cpp，由main()组成*/
# include <iostream.h>
void fun2();   //外部函数声明，等价于extern void fun2();
int n;         //全局变量定义
void main(){
  n=1;
  fun2();        // fun2()定义在文件Ex5_11_2.cpp中
  cout<<"n="<<n<<endl;}

/* Ex5_11_2.cpp，由fun2()组成*/
extern int n; //外部变量声明，n定义在文件Ex5_11_1.cpp中
void fun2() {  //fun2()被文件Ex5_11_1.cpp中的函数调用
n=3;}
```

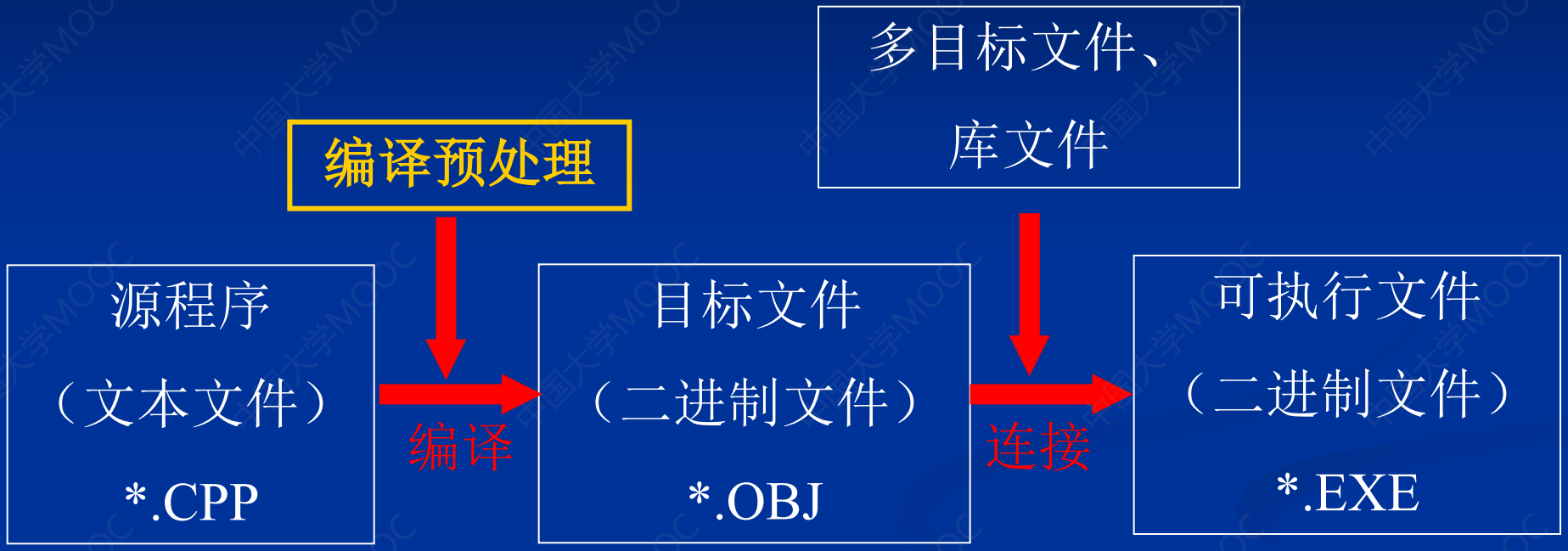
运行结果：n=3

5.12.5 存储类小结

当我们要定义使用一个变量时，应按照下面次序来确定优先：

1 非静态局部变量	函数内定义	函数内访问 函数执行生命周期	首选
2 静态局部变量static	函数内定义	函数内访问 全程生命周期	注意初始化只执行一次
3 静态全局变量static	函数外定义	本文件访问 全程生命周期	建议少用
4 非静态全局变量 外部extern变量	函数外定义	多文件共享 全程生命周期	最好不用； 建议使用 全局命名常量

5.13 编译预处理

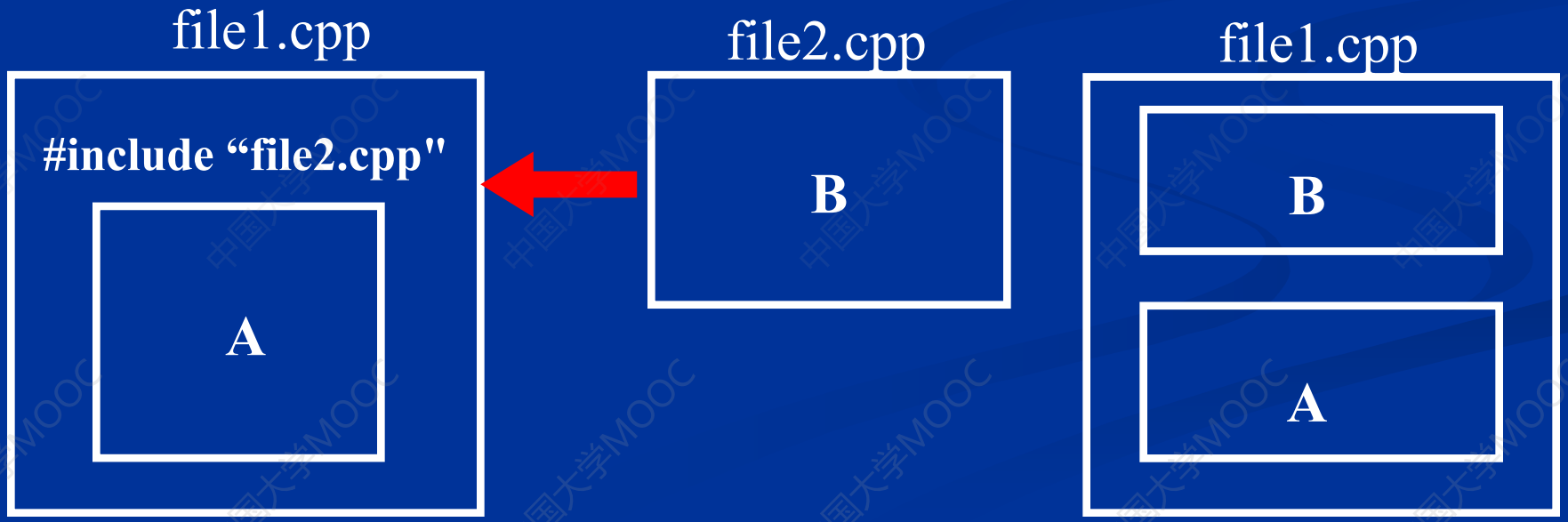


- 编译预处理指令不属于C++的语法范畴，为了区分，系统规定每条编译预处理指令用#号开头，单独占用一行。
- 编译预处理指令可分为三种：包含文件、宏和条件编译。

5.13.1 包含文件

- 编译预处理指令 include 指示将另一个源程序文件的全部内容包含 (插入) 到当前源程序文件中，替代该 include 指令行。替代后的结果写入临时文件，当前源程序文件并不改变。
- include 指令的格式为：

  # include “文件名” 或 # include <文件名>



5.13.1 包含文件

例如：C:\Program Files (x86)\Dev-Cpp\MinGW64\lib\gcc\x86_64-w64-mingw32\4.9.2\include\c++

件名表示要从当前工作目录开与源程序文件在同一个文件目录中），若找不到该文件，再到C++编译器约定的include目录中查找该文件，若仍找不到则出错。用“<”和“>”括起来的文件名表示直接从C++编译器约定的include目录中查找该文件，若找不到则出错。

- 头文件的扩展名通常为".h"，也可以使用".cpp"。
- 一条include指令只能包含一个文件。若要包含n个文件，则要使用n条include指令。
- 包含文件可以嵌套，即在一个头文件中又包含其它的头文件。
- include指令可出现在源程序中的任何一行位置，但通常放在源程序的开头。

5.13.2 无参宏

编译预处理指令 define 用来定义宏。宏可分为有参数的宏和无参数的宏。

无参数的宏

- 用一个指定的标识符（即名字）来代表一个字符串，这个标识符称为宏名。
- 编译预处理时，凡是在程序中遇到这个标识符的地方都用对应的字符串来代替，这个过程称为“宏扩展”。
- 无参宏的定义格式为：

  # define　标识符　字符串

5.13.2 无参宏

```
using namespace std;
#include<iostream>
#define  PRICE   30
int main(void)
{    int  num, total;  /*定义变量*/
      num=10;         /*变量赋值*/
     total=num*PRICE;
     cout<<"total="<<total<
}
```

编译前内部替代，作为
total=num*30

5.13.2 无参宏

- ANSI C标准定义的宏，两个下划线开头和结尾

```
__DATE__          //当前日期
__TIME__          //当前时间
__TIMESTAMP__     //当前文件的最后修改日期及时间
__FILE__          //当前文件名
__LINE__          //当前行号
```

- MS VC++添加的预定义宏

```
_M_IX86           //X86系列CPU，500表示Pentium
_WIN32            //当前平台是否为Win32，1表示是
_MSC_VER          //VC版本号，1200表示VC6.0
__cplusplus       //当前程序是否为C++编译器，1表示是
```

中国大学MOOC

5.13.2 无参宏

- 注意：
  1. 宏名通常用大写字母表示；
  2. 宏定义可出现在程序任何位置；
  3. 宏定义中可使用已定义的宏名；
  4. 宏扩展时对宏对简单的代换，不作任何计算；
  5. 可使用预处理命令 #undef 宏名 终止宏名作用域；
  6. 当宏名出现在字符串中，作为字符处理；
  7. 同一作用域中，同一宏名只能定义一次。

单选题 1分

设置

设有以下宏定义和语句，则i 的值为 。

```
#define ONE 1
#define TWO (ONE+ONE)
#define THREE ONE+TWO
i=THREE*3+TWO*2;
```

A 13

B 11

C 9

D 8

提交

中国大学MOOC

5.13.3 有参宏

#define　宏名(参数表)　字符串

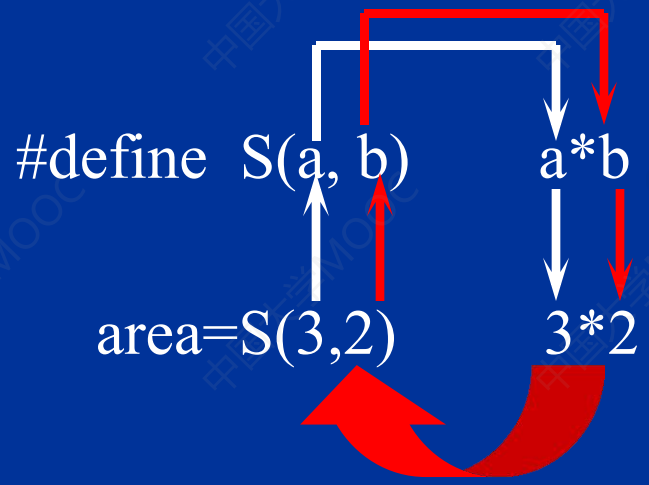
```
#define      S(a, b)              a*b
....
area=S(x, y);    /*  area=x*y;  */
```

形式参数

实际参数

5.13.3 有参宏

- 按#define命令行中指定的字符串从左至右进行置换宏名，字符串中的形参以相应的实参代替，字符串中的非形参字符保持不变。



5.13.3 有参宏

```
using namespace std;
#include<iostream>
#define  PI  3.1415926
#define   S(r)   PI*r*r
int main( )
{
   float  a,  area;
   a=2;
   area=S(a+2);
   cout<<"r="<<a<<"area="<<area<<endl;
   return 0;
 }
```

#define S(r) PI *r*r 换成：
#define S(r)   PI*(r)*(r)

area=3.1415926*(a+2)*(a+2)
实际效果

a=3.1415926*a+2*a+2
实际效果

注意：

(1) 实参有可能是表达式时,在宏定义中要用括号将形参括起来。

(2) 在定义带参宏时, 宏名与左圆括号之间不能有空格。

(3) 当一个宏定义多于一行时，在按换行符（Enter键）之前，必须先使用转义字符“\”。

(4) 宏与函数的定义形式不一样。宏定义中只列出形参名，不说明形参的类型。

(5) 宏调用时，不对参数作任何计算，只作简单的替换。

(6) 宏调用时，不对参数作任何检查。

(7) 宏没有返回值的概念。

(8) 多次调用同一个宏，经宏替换后，会增加程序的长度。

单选题 1分

设置

下面程序的编译运行结果是

```
#define AA 10
#define D  (x)  x * x //A
int main(void){
  int x = 1, y = 2, t;
  t = D(x+y) * AA; //B
  cout<<t;
  return 0;
}
```

A 编译出错

B 50

C 30

D 23

提交

中国大学MOOC

5.13.4 条件编译

```
#define  DEBUG
int main( )
{ int  a=14, b=15,  c;
  c=a/b;
  # ifdef  DEBUG
  cout<<“a=”<<a;
  cout<<" b="<<b<<endl;
  # endif
  cout<<"c="<<c<<endl;
  return 0;
}
```

```
int main( )
{ int  a=14, b=15,  c;
  c=a/b;

  cout<<“a=”<<a;
  cout<<" b="<<b<<endl;

  cout<<"c="<<c<<endl;
  return 0;
}
```

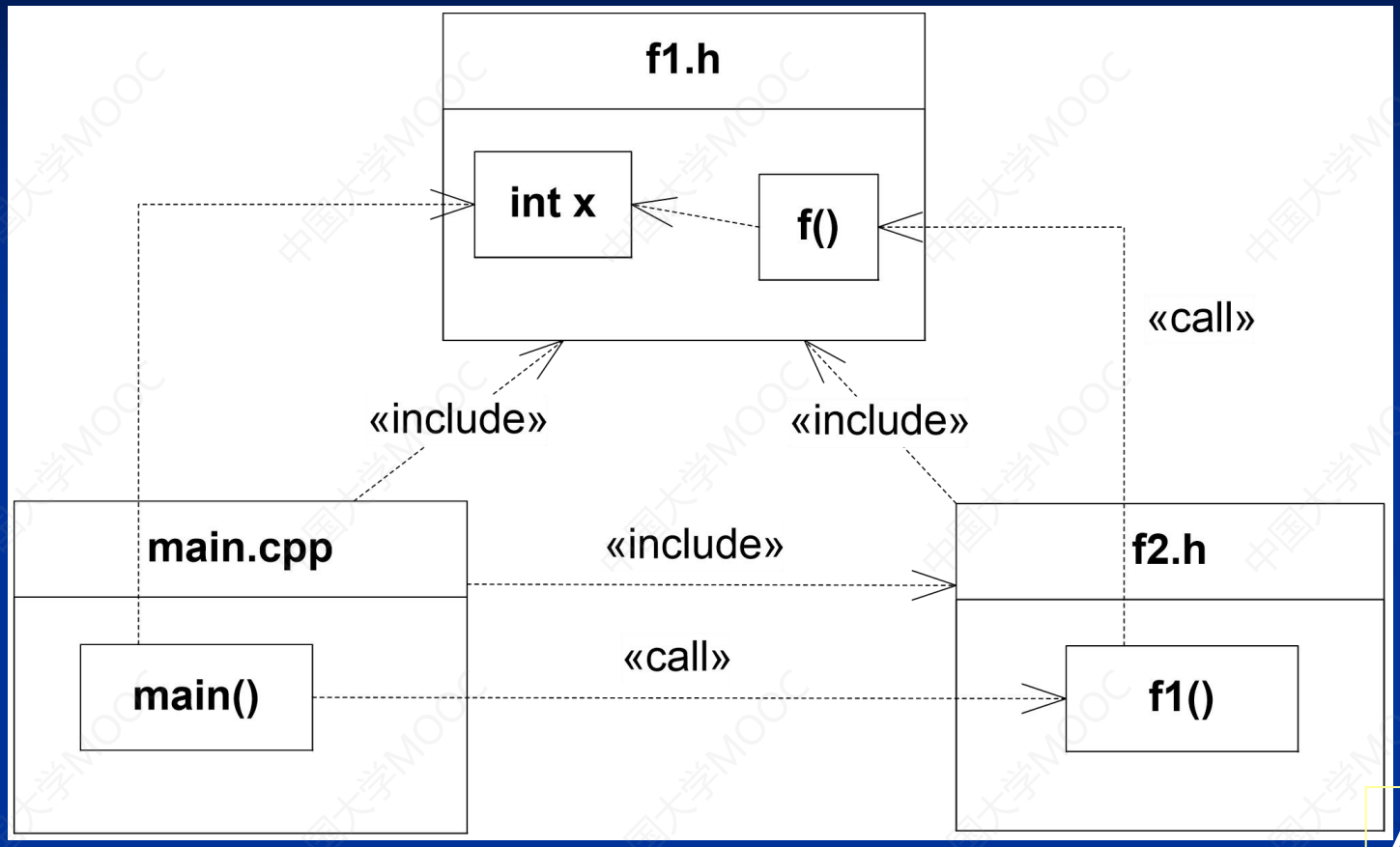
5.13.4 条件编译

```
int main( )
{ int  a=14, b=15,  c;
  c=a/b;
  # ifdef  DEBUG
  cout<<“a=”<<a;
  cout<<" b="<<b<<endl;
  # endif
  cout<<"c="<<c<<endl;
  return 0;
}
```

```
int main( )
{ int  a=14, b=15,  c;
  c=a/b;


  cout<<"c="<<c<<endl;
  return 0;
}
```

5.13.5 条件编译示例



同一个头文件f1.h被多次包含

导致x和f()重复定义

```
//f1.h
int x = 11;
void f(){
    x += 2;
}
```

5.13.5 条件编译示例

- 合理使用条件编译可避免对同一个头文件的多次包含。

```
#ifndef F1_h
#define F1_h
    … //头文件代码
    …
#endif
```

5.13.6 其它预处理指令

- #error <字符串>
  - 产生编译错误信息，将字符串作为错误信息并停止编译。
- #line <正整数> “文件名”
  - 更改内部存储的当前文件名和当前行。
- #pragma <指令>
  - 为当前编译器提供一种控制编译的方式，以保持与C/C++的兼容性。
- #import <文件名> [属性表]
  - 从一个类型库文件中导入信息，转换为C++类，并生成两个头文件，用C++圆满重新构建类型库内容。